CSci 23: Module 1 Applications Development →

Managing Repositories

*

Version: v3.2026

Rodney Maniego Jr.
Department of Computer Science and Technology
Faculty of Computing * Visayas State University

Disclaimer: This presentation is specially prepared as part of the CSci 23 Applications Development and Emerging Technologies course for the Visayas State University. It is provided for educational purposes under fair use. Unauthorized copying, distribution, or reproduction of this material, in whole or in part, is strictly prohibited.

*

Course Outcome

Manage application development using version control.

Learning Outcomes

1. Explain the fundamental concepts about version control.
2. Manage repositories with multiple collaborators.

Outline

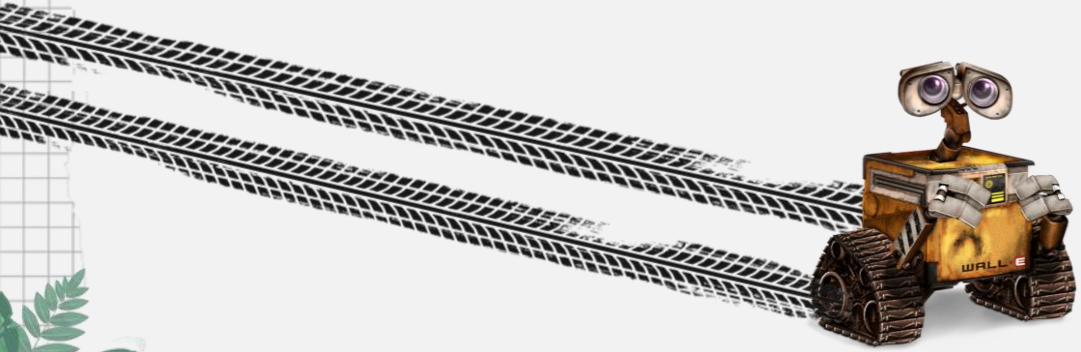
1. Design with Reuse
2. Versioning
3. Version Control System
4. Setting Up Git
5. Git Concepts
6. Basic Git Commands
7. Advanced Git Commands
8. Pull Requests
9. Commit Messages

Managing Repositories

Design with Reuse

What is Design by Reuse?

It is a software engineering approach in which new systems are designed and constructed using existing assets instead of building entirely from scratch.



What is Design by Reuse?

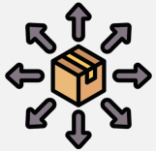
#1 Application System Reuse

It maximizes system-wide reuse with no change in its subsystems through specialized configurations or by developing application families that are specialized for specific customers or platforms.

What is Design by Reuse?

#2 Component-based Reuse

It promotes the reuse of components ranging from small objects to large subsystems into a separate application or system.



What is Design by Reuse?

#3 Function Reuse

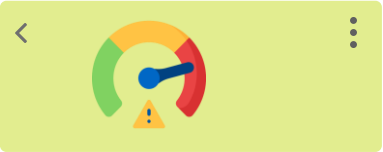
It implements a set of functions that can be reused into different software projects; this is more popularized by standard and third-party libraries shared in open-source communities.

What is Design by Reuse?

Advantages

Reliability

Reusing heavily tested components should be more reliable than newly developed components.



Reduced Risk

Reusing existing components provide less costs and uncertainties versus the cost of development.

Compliance

UI/UX standards, when applied to reusable components provide less likelihood of errors and mistakes.

Acceleration

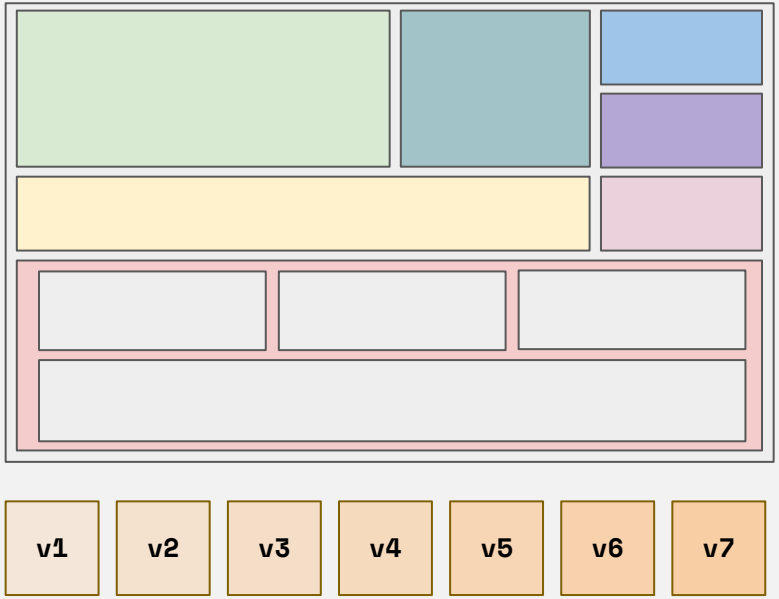
Reuse accelerates development which also reduces validation time and time-to-market.

Architecture

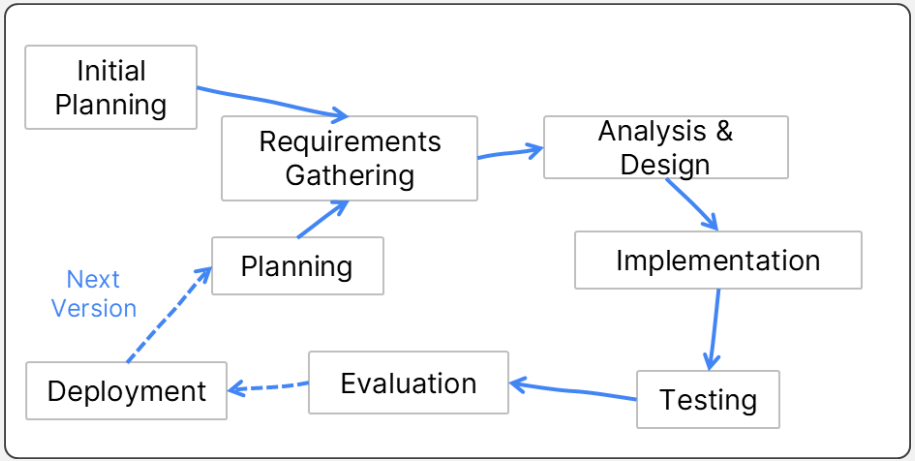
It is a high-level design and structure of a software.

Architecture

Software Architecture



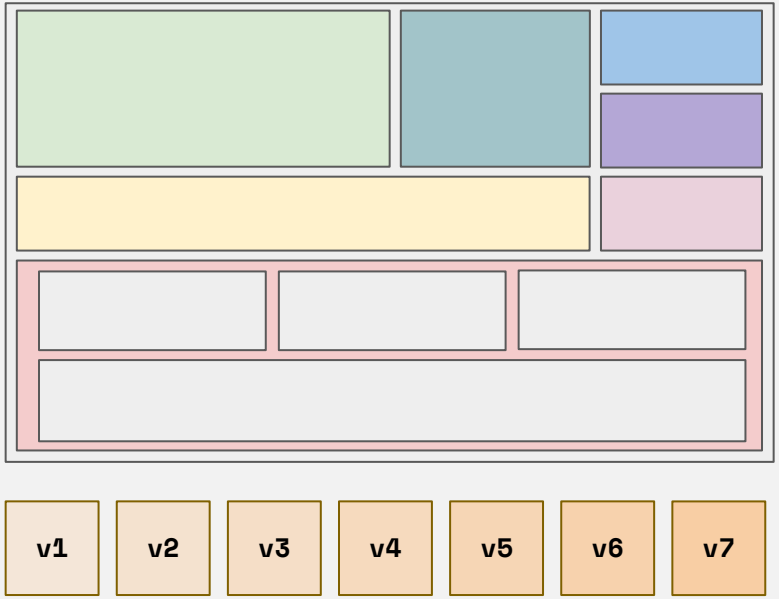
Major Versions
(Evolutionary Model)

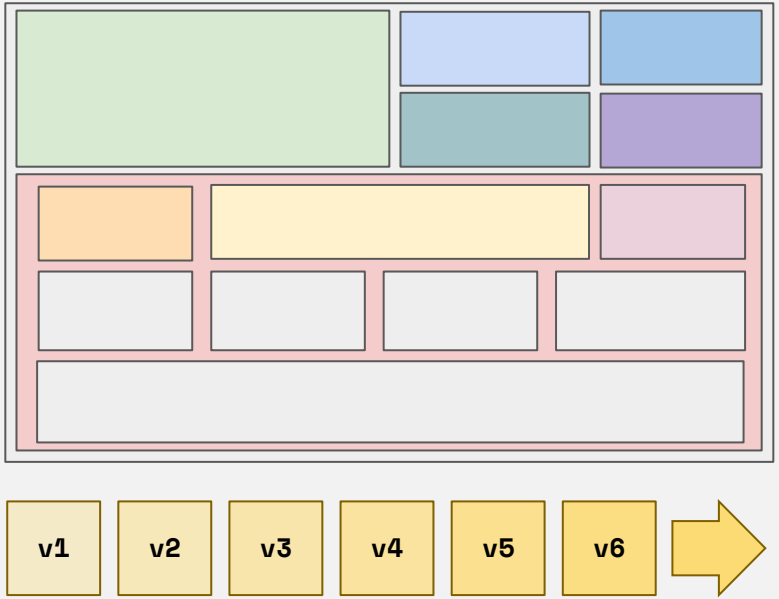


* It is like iterative-incremental model, but each iteration is a complete version.

Architecture

“Legacy” Software Architecture → "New" Software Architecture

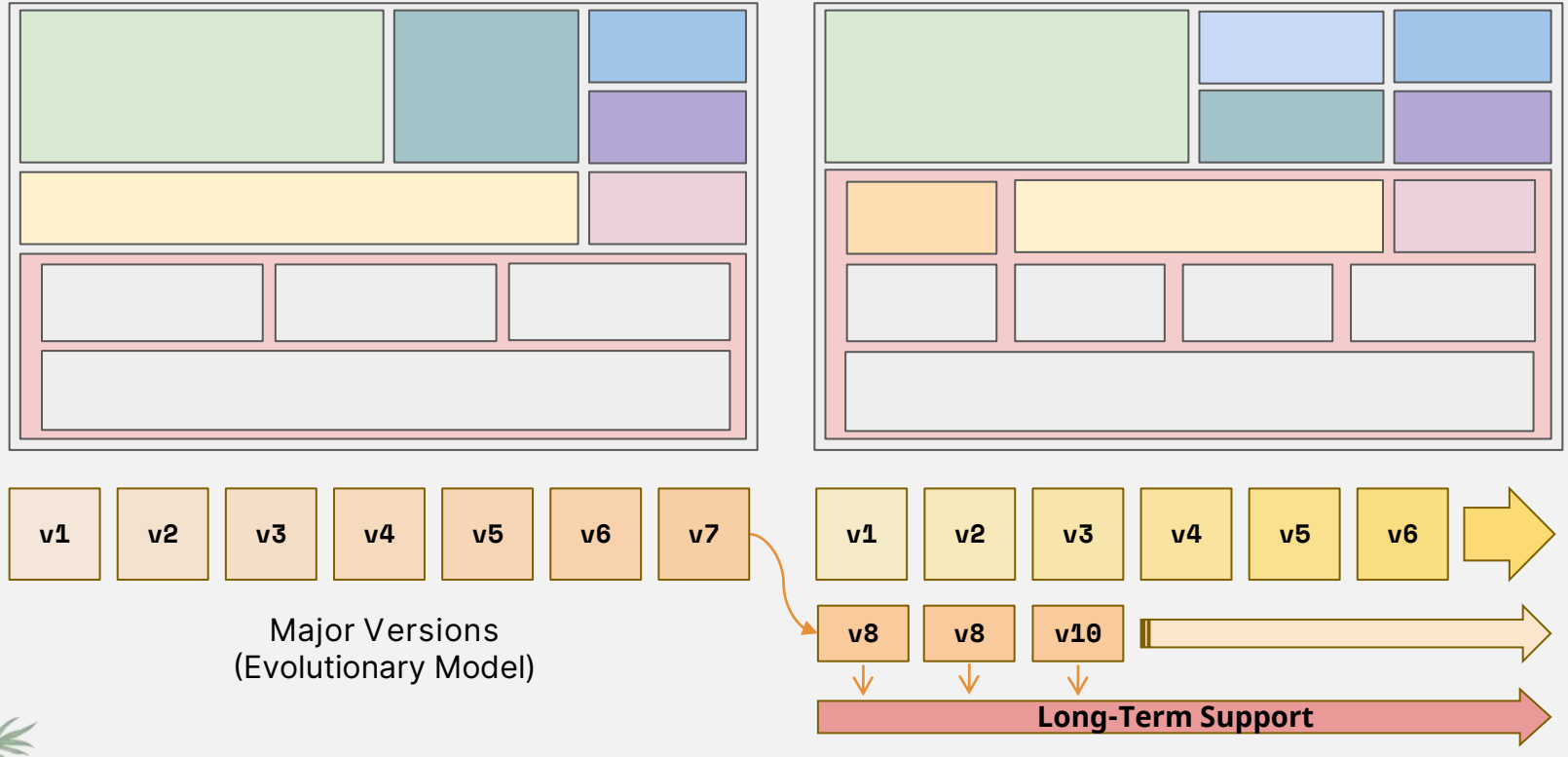




Major Versions
(Evolutionary Model)

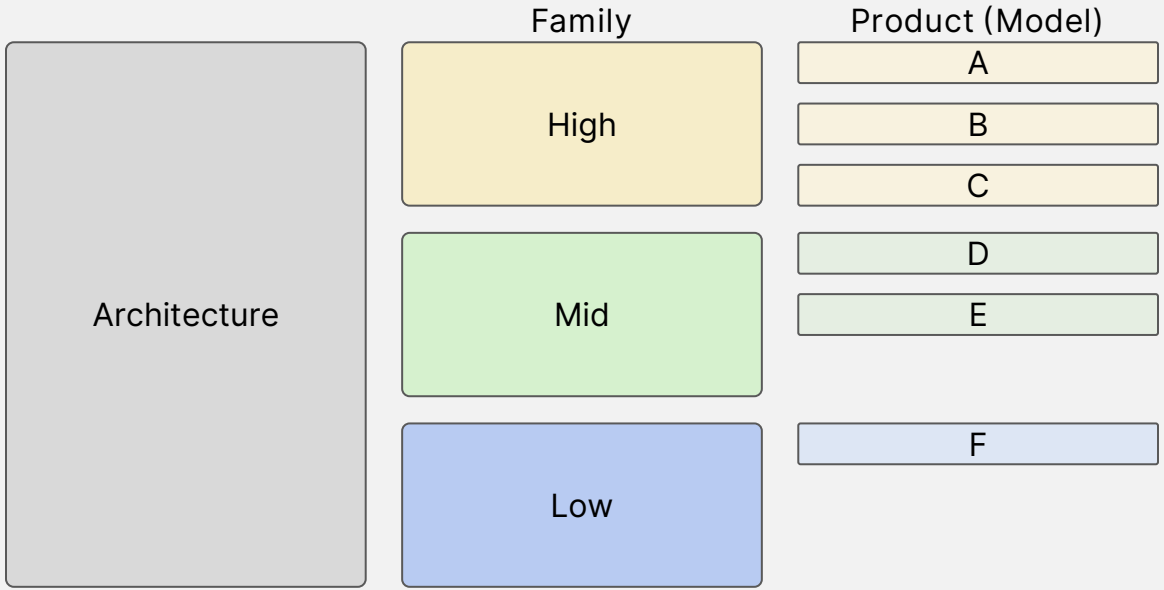
Architecture

“Legacy” Software Architecture → "New" Software Architecture



Architecture

Product Line



Managing Repositories

Versioning

Semantic Versioning

It is a set of guidelines on how software versions are assigned and incremented.

x	y	.	w
major	minor		patch
Any incompatible changes.	New (compatible) functionality.		Bug fixes and small changes.

Semantic Versioning

Major Branch

It is a long-lived branch representing a primary line of development aligned with a strategic product generation.

010 .001

major	minor	patch
Any incompatible changes.	New (compatible) functionality.	Bug fixes and small changes.

Semantic Versioning

Internal Release

It is a build intended for internal software verification and validation.

010 .001*

major	minor	patch
Any incompatible changes.	New (compatible) functionality.	Bug fixes and small changes.

* Each patch increment may contain one or more changes or fixes; certain organizations may build weekly releases. Software testing will run in parallel with the software development.

Semantic Versioning

Golden Code

The internal version that passed testing and review.

010 .036

major	minor	patch
Any incompatible changes.	New (compatible) functionality.	Bug fixes and small changes.

* After external release, no more changes are made in this branch.

Semantic Versioning

Maintenance Branch

It is used for incremental fixes to address critical or urgent defects in the latest release version.

011.001

major	minor	patch
Any incompatible changes.	New (compatible) functionality.	Bug fixes and small changes.

Semantic Versioning

Maintenance Branch – Golden Code

It is candidate code from the maintenance branch containing fixes but without new features.

011 .017

major	minor	patch
Any incompatible changes.	New (compatible) functionality.	Bug fixes and small changes.

Semantic Versioning

Next Major Branch

It is a long-lived branch used to develop the next product generation that will replace the current major branch.

020 .001

major	minor	patch
Any incompatible changes.	New (compatible) functionality.	Bug fixes and small changes.

Internal Codenames

It is a practice of using unique names to simplify the management of legacy and parallel projects while also providing confidentiality and focus.

CN	.	x	y	.	w
codename		major	minor		patch
A secret name assigned to a service or product.		Any incompatible changes.	New (compatible) functionality.		Bug fixes and small changes.

Internal Codenames

Advantages

Confidentiality

Prevents leaking of sensitive information of under-development products by adding a layer of ambiguity.

Differentiation

Allows short and memorable differentiation in products delivered simultaneously.

Independence

Development teams can focus more on their goals while the marketing team develop the branding.

Identity

Thematic naming adds cultural impact and improves organizational camaraderie.

Internal Codenames

Step #1: Choose a Theme

Internal Codenames

Step #2: Choose a Codename

🐎 DON QUIXOTE

🌊 Fiji

🌕 Feldspar

🌌 TARANTULA

🦚 Sarimanok

Internal Codenames

Step #3: Choose a Short Code

🐎 DQ

🌊 Fd

🌕 7S

🌌 TR

🦚 SM

Internal Codenames

Step #4: Format into Release Codes

🐎 DQ.010.001

🌊 FJ.010.001

🌕 7S.010.001

🌌 TR.010.001

🦚 SM.010.001

Internal Codenames

Step #5: Apply Versioning Rules

Internal Release

SM.010.001

SM.010.002

∗ minor changes + bug fixes

Internal Codenames

Step #5: Apply Versioning Rules

SM.010.001

Internal Release

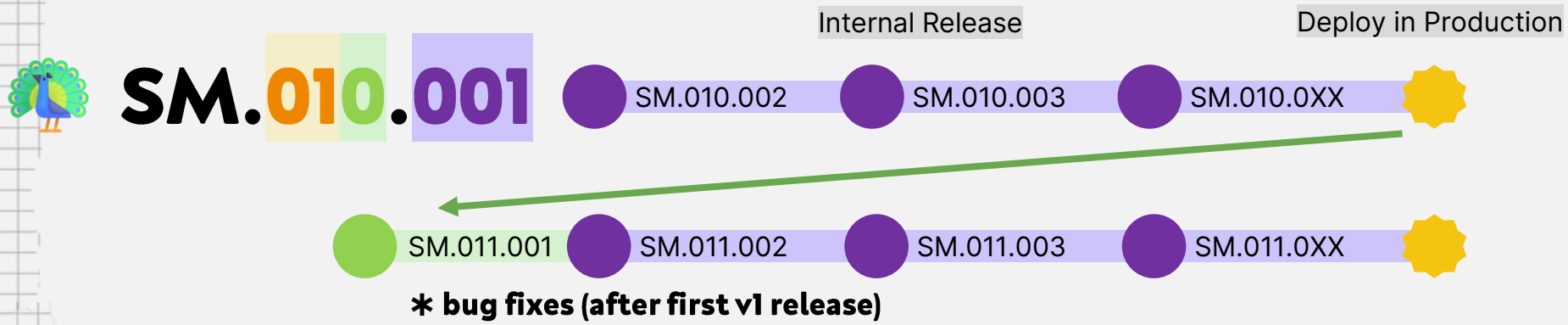
Deploy in Production

SM.010.002 — SM.010.003 — SM.010.0XX

∗ minor changes + bug fixes

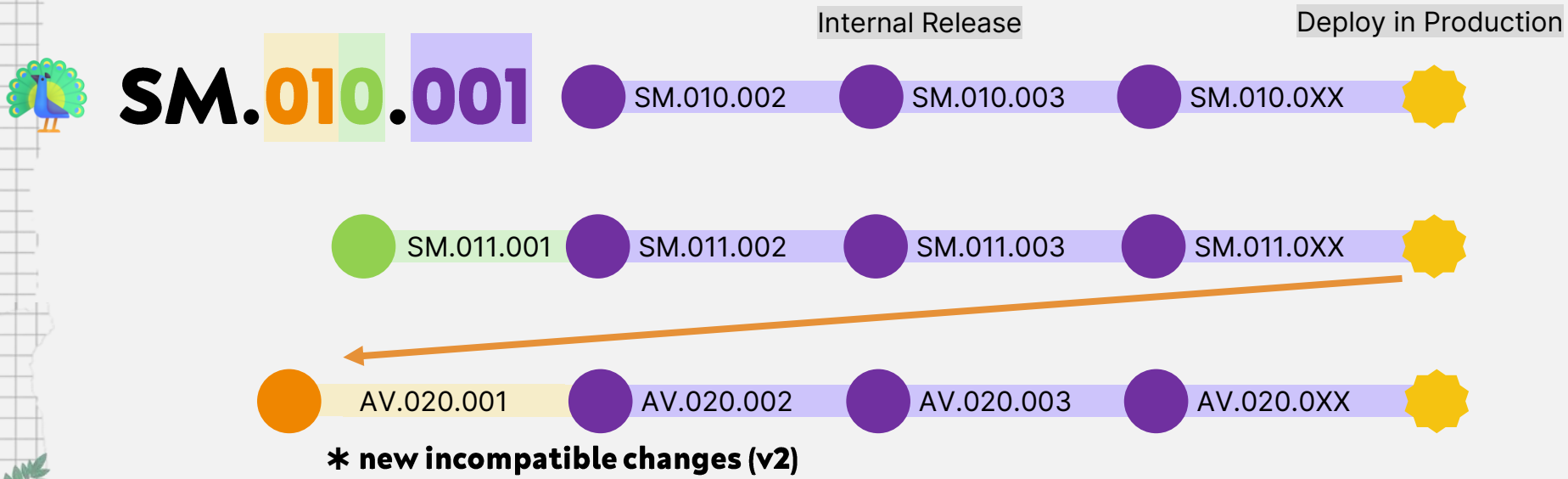
Internal Codenames

Step #5: Apply Versioning Rules



Internal Codenames

Step #5: Apply Versioning Rules



Internal Codenames

Step #5: Apply Versioning Rules

* bug fixes (after first v2 release)

Managing Repositories

Version Control System

Project-Related Challenges

1. Codes get convoluted as it grows.
2. Offline collaborative work is challenging.
3. Duplicate files takes so much space.
4. Manually tracking changes and issues is hard.
5. Files can be corrupted or lost.

What is a Configuration?

It is the arrangement of the HW/SW of a system; its management ensures greater control over the system while making sure the system functions smoothly.

Configuration Management

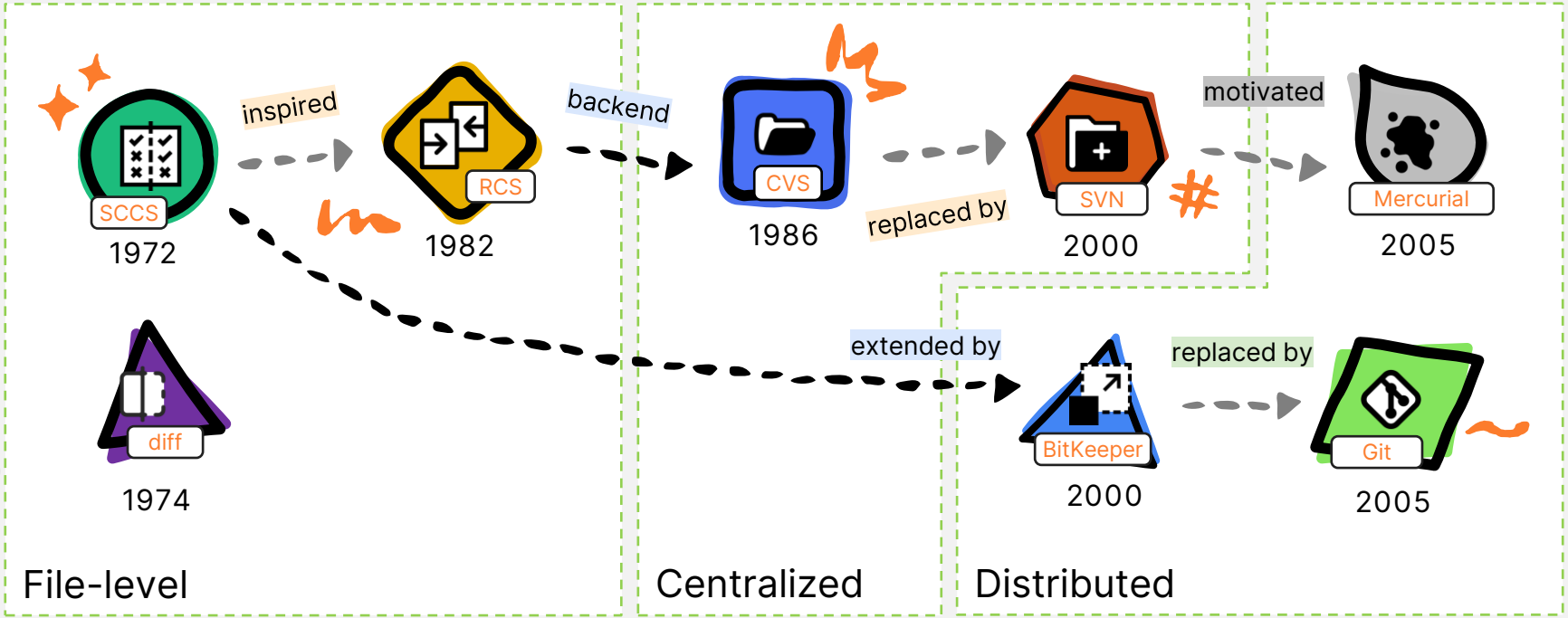
It originates from the US Department of Defense as a technical management discipline for handling hardware materials and changes systematically, so it maintains its integrity over time.

What is SCM?

A Configuration Management *for* Software.

It is a methodology for change requests and tracking, version control, security administration, software quality reviews, and software distribution.

SCM History



* Every clone contains the entire repository and history.

What is a Version Control System?

It is a system that tracks the history of changes as people collaborate on a project while also allowing the recovery of any earlier versions.

https://docs.github.com/en/get-started/using-git/about-git

What is a Version Control System?

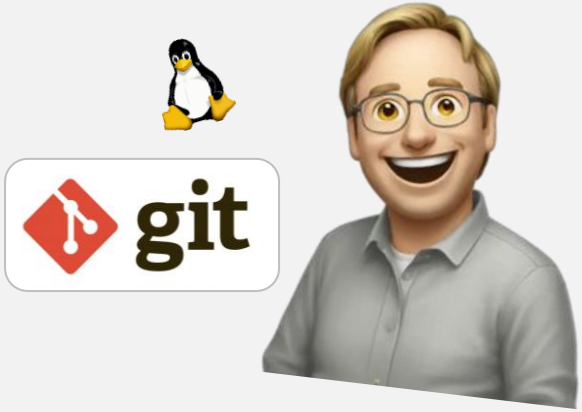
Developers can find out:

- Which changes were made?
- Who made the changes?
- When were the changes made?
- Why were changes needed?

https://docs.github.com/en/get-started/using-git/about-git

What is Git?

Is an open-source, distributed source code management system with strong safeguards against accidental and malicious corruptions.



https://docs.github.com/en/get-started/using-git/about-git

Advantages of Git

Distributed Development
It supports local, centralized, and distributed environments.

Non-linear Development
It allows branching and merging.

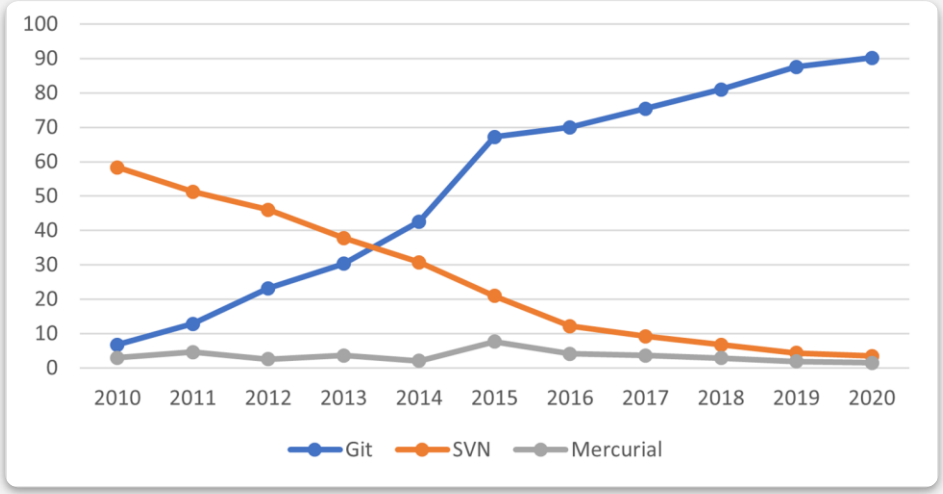
Protocol Compatibility
It supports both HTTPS and SSH for diverse types of technical and non-technical users.

Security and Traceability
It ensures the integrity and traceability of the commit history through cryptographic authentication.

Fast and Scalable
It performs most operations locally and updates are done remotely on-demand.

https://docs.github.com/en/get-started/using-git/about-git

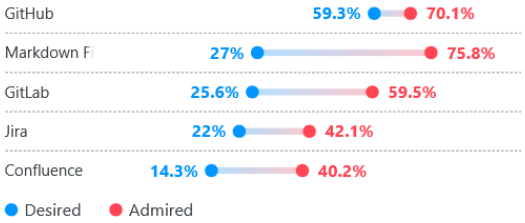
Popularity of Git



2026 Developer Survey

Technology → Admired and Desired

GitHub is a more desirable collaboration tool than Jira this year



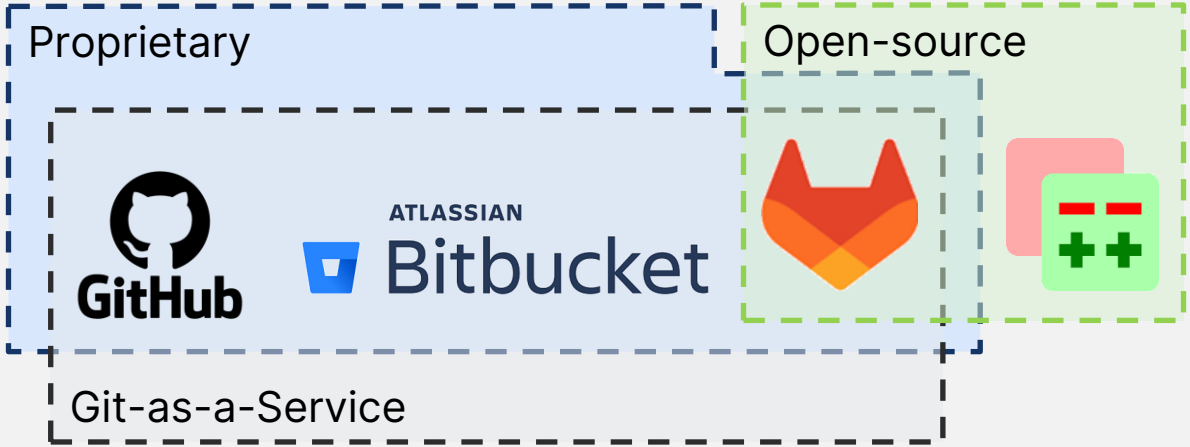
Jira steps down as the most desired tool for code documentation and collaboration and the new top desired tool is GitHub. Markdown continues to be the most admired sync tool for the third year.

Code documentation and collaboration tools →

https://survey.stackoverflow.co/2022
https://survey.stackoverflow.co/2025

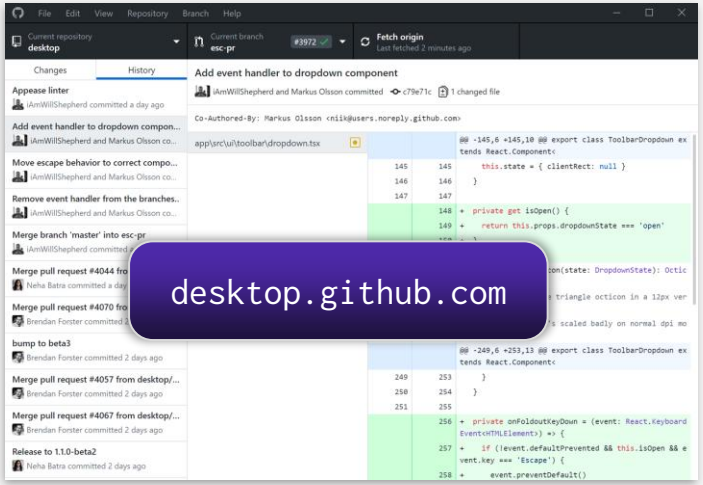
SCM Platform

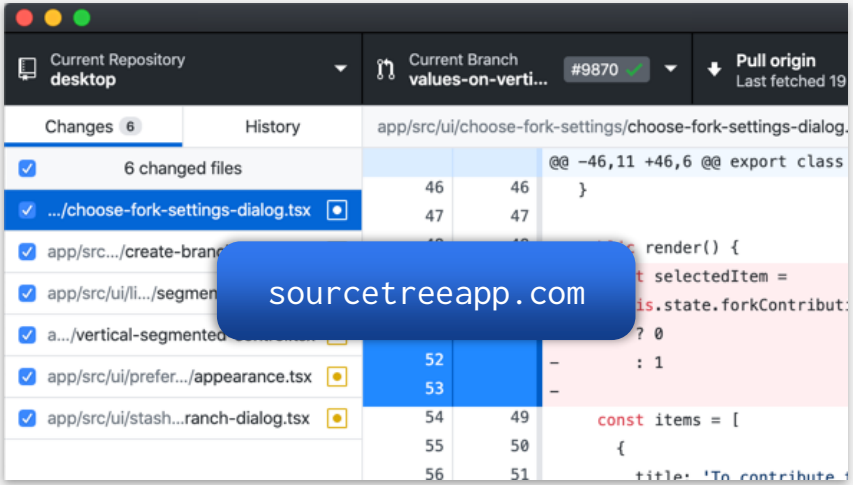
It is a platform enabled by Git that provides other functions such as access management, code review, issue tracking, and CI/CD pipelines.



Git Clients

It includes several third-party tools for users looking for platform-specific experience.



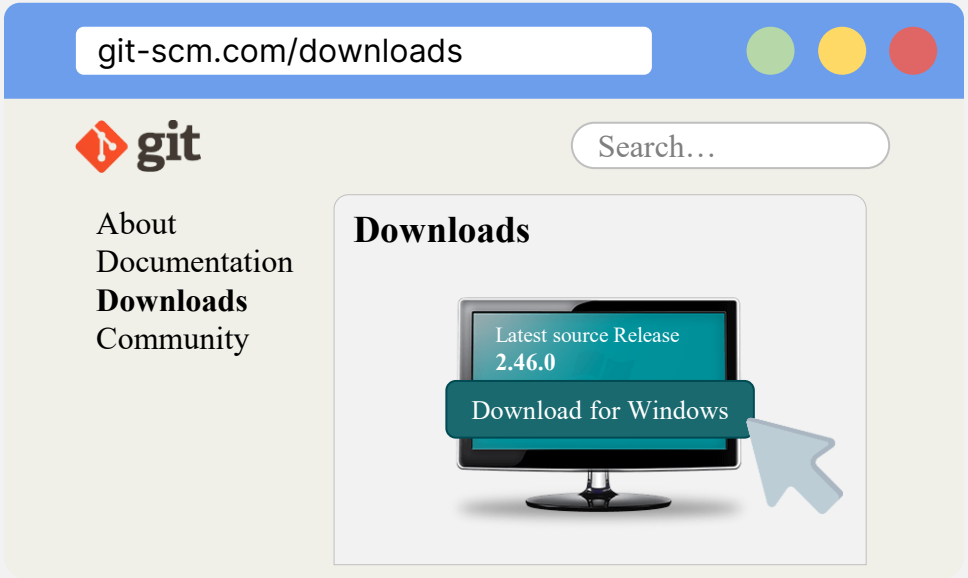


Managing Repositories

Setting Up Git

Installing Git...

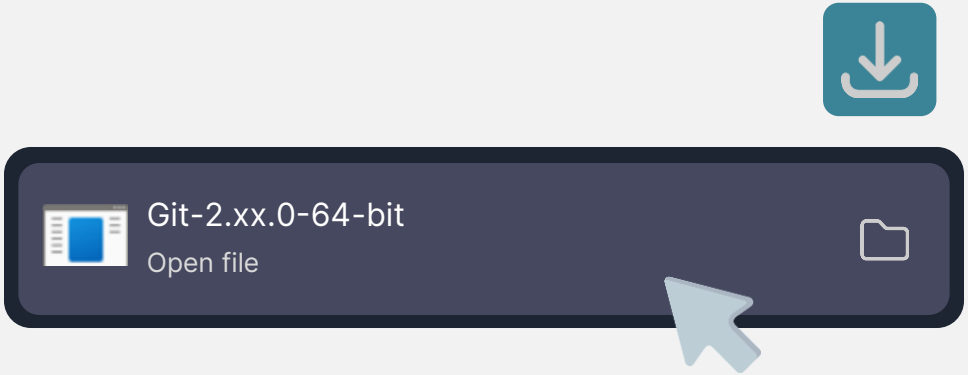
#1



NOTE: This is for first time installation only, other platform may also require a different installation process.

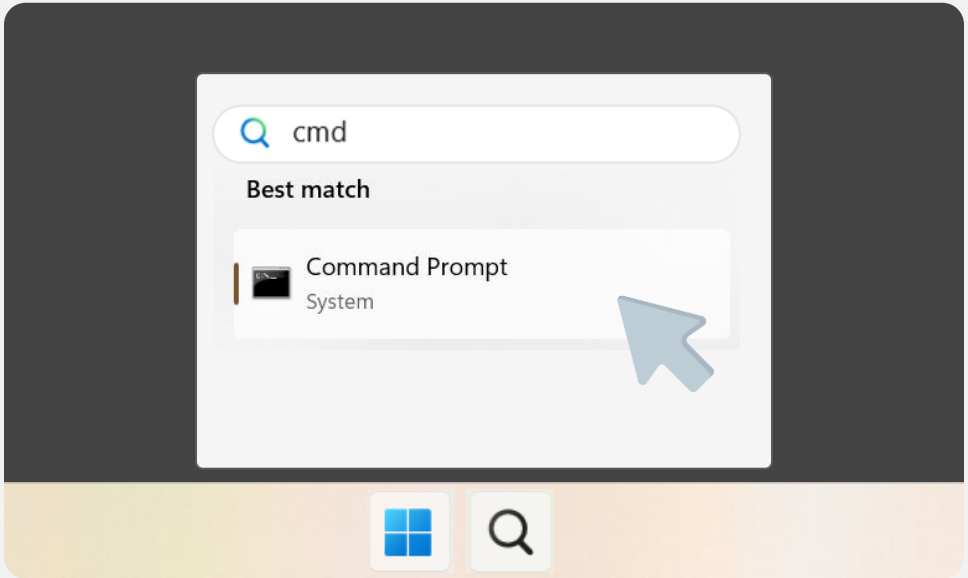
Installing Git...

#2



Installing Git...

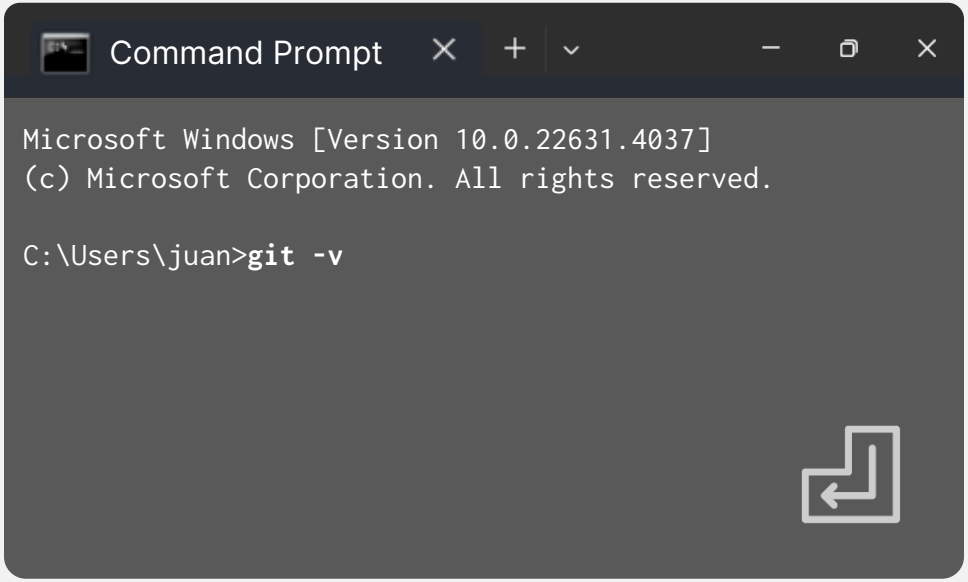
#3



NOTE: Command Prompt is pre-installed on Windows.

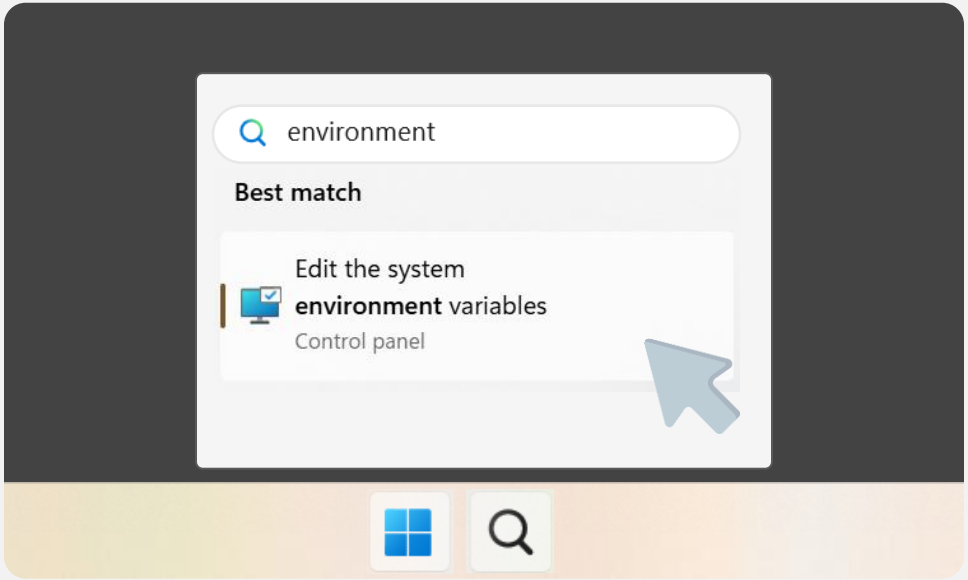
Installing Git...

#4



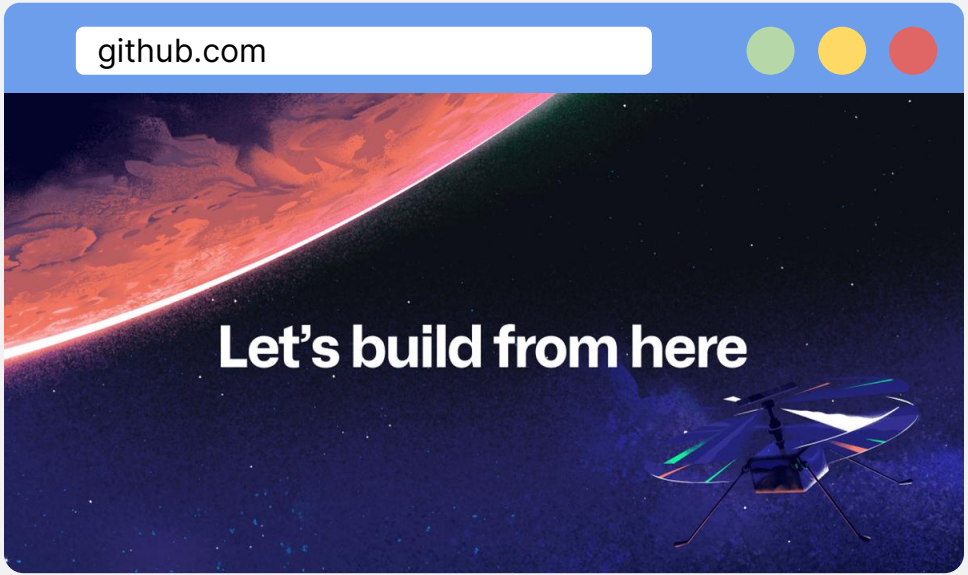
If not recognized...

#5



Setting Up Git for Windows

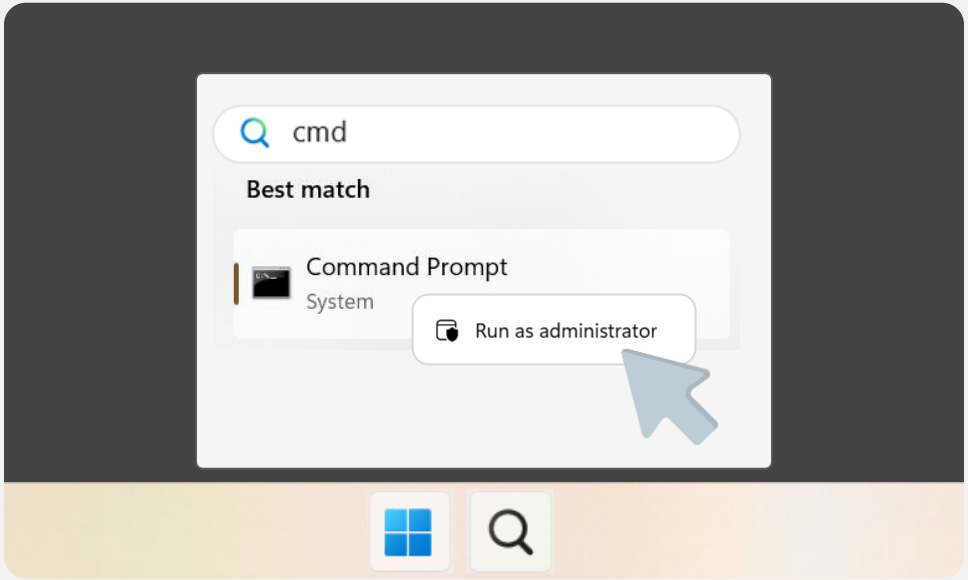
#1



NOTE: Sign up or sign in first to start managing account.

Setting Up Git for Windows

#2



Setting Up Git for Windows

#3

Command Prompt

```
C:\Users\juan>cls

C:\Users\juan>git config --global user.name "Juan dela Cruz"

C:\Users\juan>git config --global user.email "jdcruz@website.com"
```

NOTE: Do only once per PC or user.

Setting Up Git for Windows

#4

Command Prompt

```
C:\Users\juan>cls

C:\Users\juan>ssh-keygen -t ed25519 -C jdcruz@example.com

C:\Users\juan>Enter file in which to save the key: [ENTER]

C:\Users\juan>Overwrite (y/n)? y
```

NOTE: Do only once per PC or user.

Setting Up Git for Windows

#5

Command Prompt

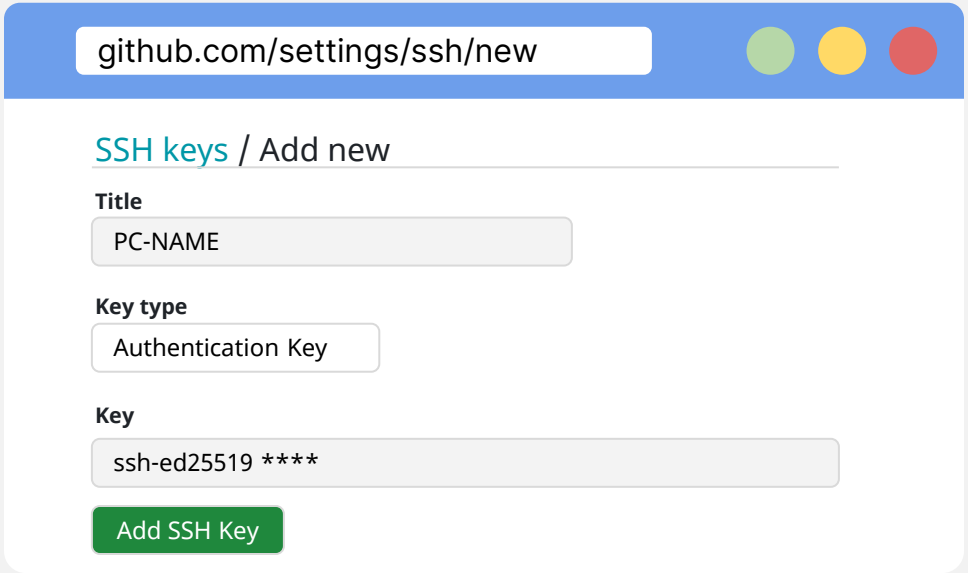
```
C:\Users\juan>cls

C:\Users\juan>type "C:\Users\%USERNAME%\.ssh\id_ed25519.pub"
ssh-ed25519 ****

C:\Users\juan>
```

Setting Up Git for Windows

#6



NOTE: Sign up or sign in first to start managing account.

Setting Up Git for Windows

#7

config

```
Host github.com
User git
Hostname ssh.github.com
PreferredAuthentications publickey
IdentityFile ~/.ssh/id_ed25519
Port 443
```

NOTE: Enable SSH-based Git operations (clone, push, pull) even in restricted networks, by forcing connections to port 443 when port 22 is blocked.

Setting Up Git for Windows

#8

Command Prompt

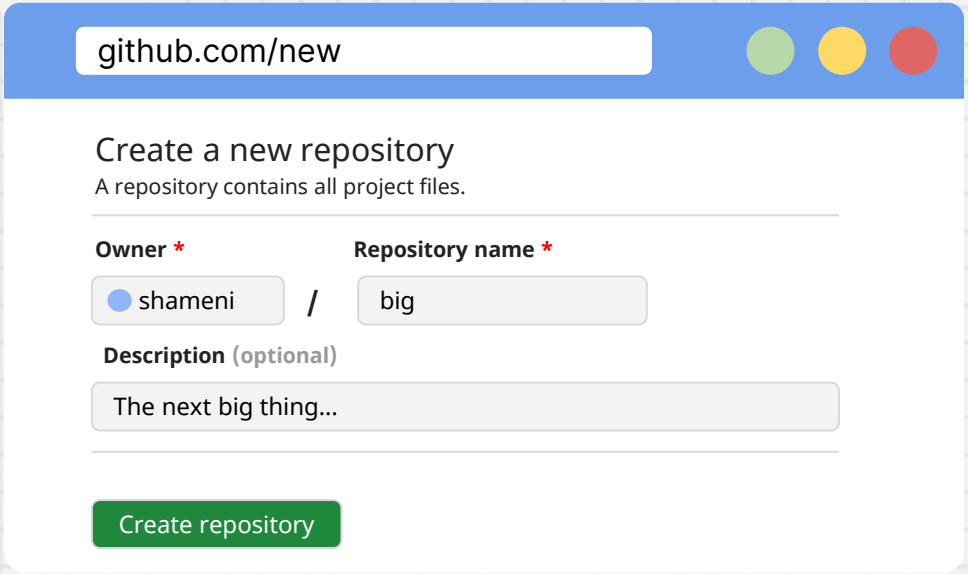
```
C:\Users\juan>cls

C:\Users\juan>ssh -T git@github.com
Warning: Permanently ... list of known hosts.

C:\Users\juan>
```

Setting Up Git for Windows

#9



NOTE: Set to `private` as necessary.

Managing Repositories

Git Concepts

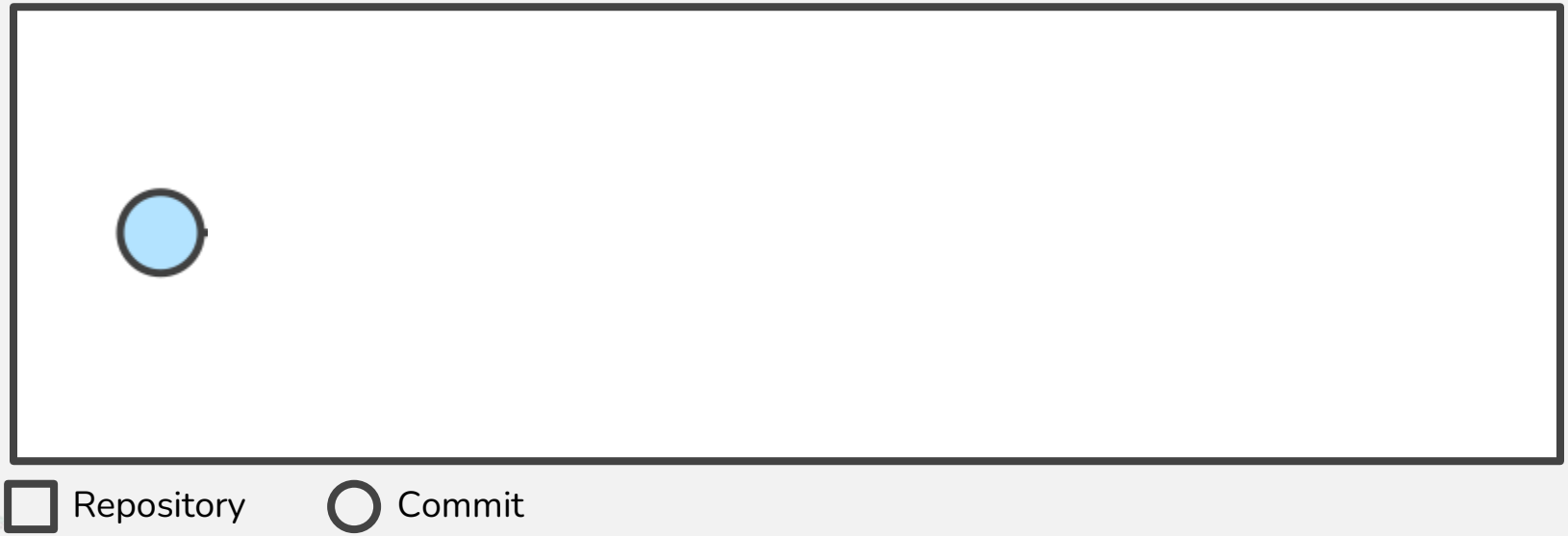
Repository

It is content-addressed object database of structured data, that is indexed and queried by the Git engine.

Repository

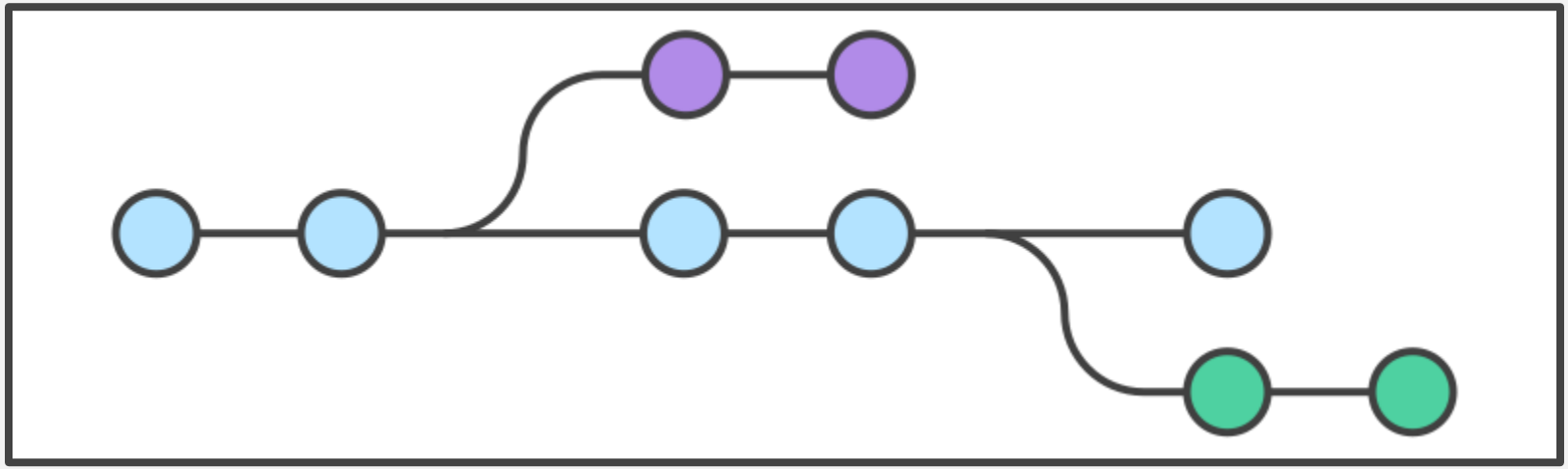
Commit

It is an immutable object that records a snapshot of the repository state, along with metadata and parent references.



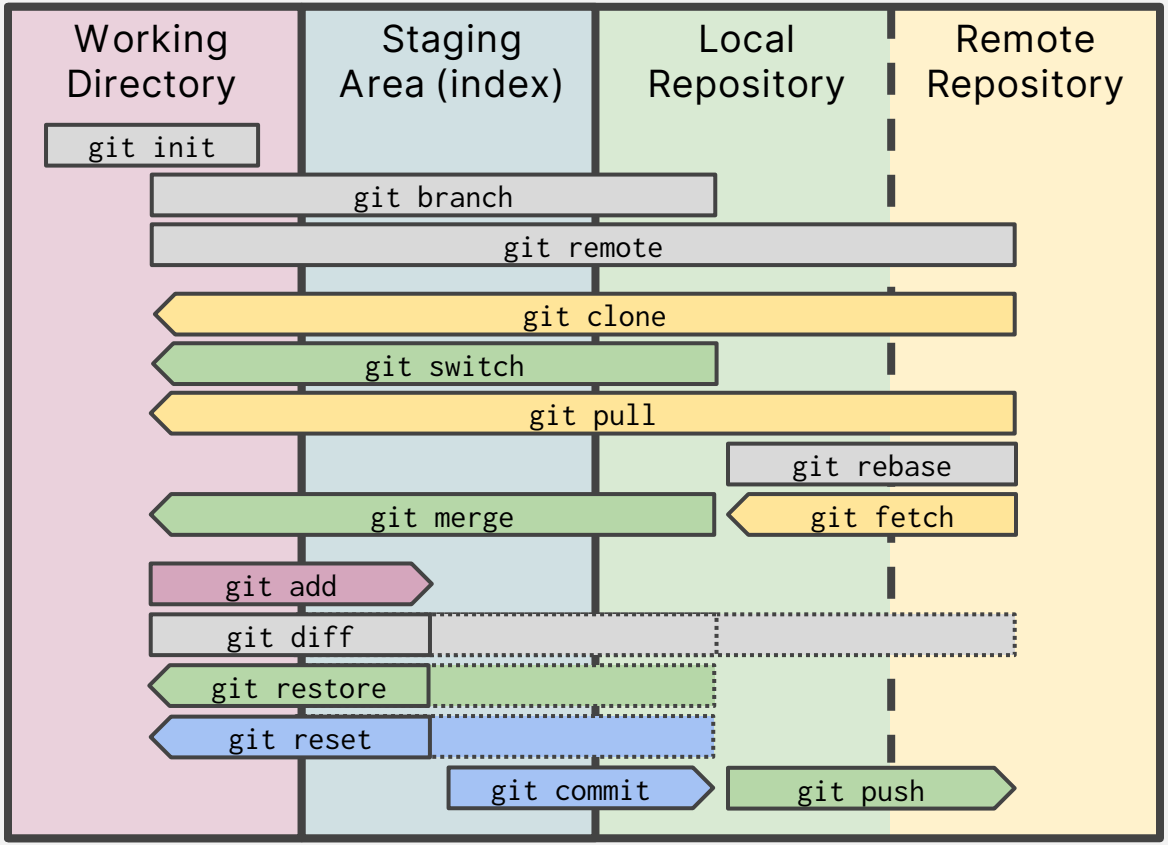
Branch

It is a movable reference that points to a commit, representing a line of development.



Repository Commit Branch

Git Workflow



Managing Repositories

Basic Git Commands

Initialize and Point to Remote Repo

#1

Command Prompt

```
C:\Users\juan>cd /d "c:/projects/big"

C:\projects\big>git init

C:\projects\big>git commit -F .commits/message.txt

C:\projects\big>git branch -M main

C:\projects\big>git remote add origin git@github.com:shameni/big.git

C:\projects\big>git push -u origin main
```

Point Old Repo to Empty Repo

#2

Command Prompt

```
C:\Users\juan>cd /d "c:/projects/old"

C:\projects\old>git remote set-url origin git@github.com:shameni/big.git

C:\projects\old>git add .

C:\projects\old> git commit -F .commits/message.txt

C:\projects\old>git push -u origin main
```

Cloning A Repository

#3

Command Prompt

```
C:\Users\juan>cd /d "c:/projects"

C:\projects\>git clone git@github.com:shameni/monnke.git

C:\projects\>cd monnke

C:\projects\monnke>git add <./*/folder/file/regex>

C:\projects\monnke>git commit -F .commits/message.txt

C:\projects\monnke>git push -u origin main
```

Display Unstaged Changes

#4

Command Prompt

```
C:\projects\monnke>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
        modified:   architectures.py

no changes added to commit (use "git add" and/or "git commit -a")
C:\projects\monnke>
```

Adding Changes to Staging Area

```
git add filepath
```

Stages specific file.

```
git add folder/
```

Stages all files in the specified folder.

```
git add *.jpg
```

Explicitly stages all JPEG files in current folder.

```
git add -A
```

```
git add -all
```

Stages all new, modified, and deleted files in the whole working directory.

```
git add -u
```

```
git add -update
```

Stages all modified and deleted files only.

```
git add .
```

Stages all new, modified, and deleted files in the current folder.

```
git add *
```

Stages all (visible) new, modified, and deleted files in the current folder.

https://git-scm.com/docs/git-add

Unstage Folder/File

#6

Command Prompt

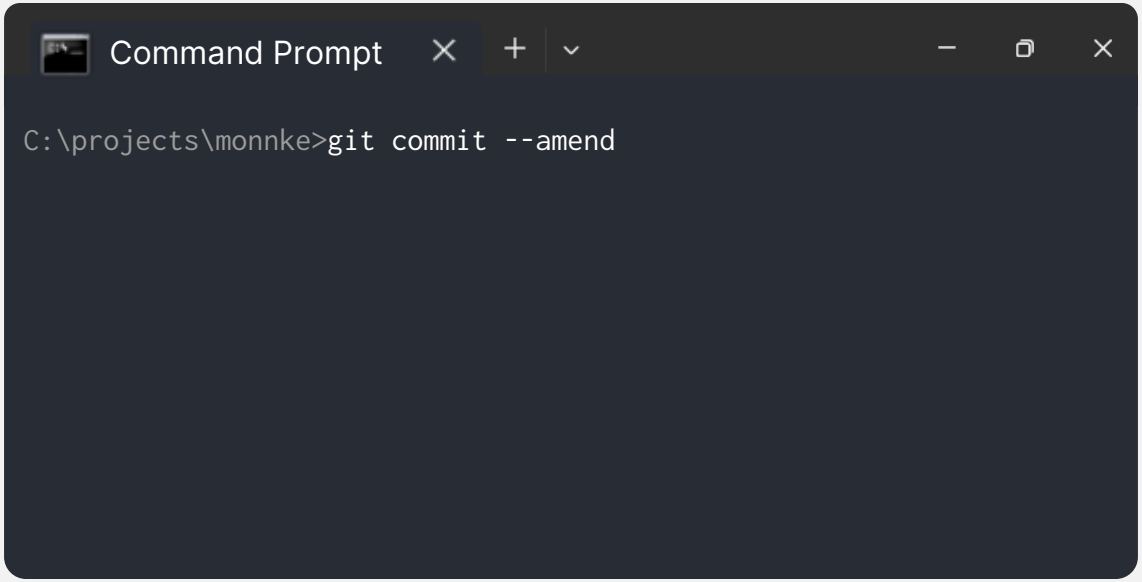
```
C:\projects\monnke>git add models/

C:\projects\monnke>git reset models/*.keras
```

NOTE: Avoid using `git restore` as it will effectively overwrite changes in the working directory.

Change Recent Commit Message

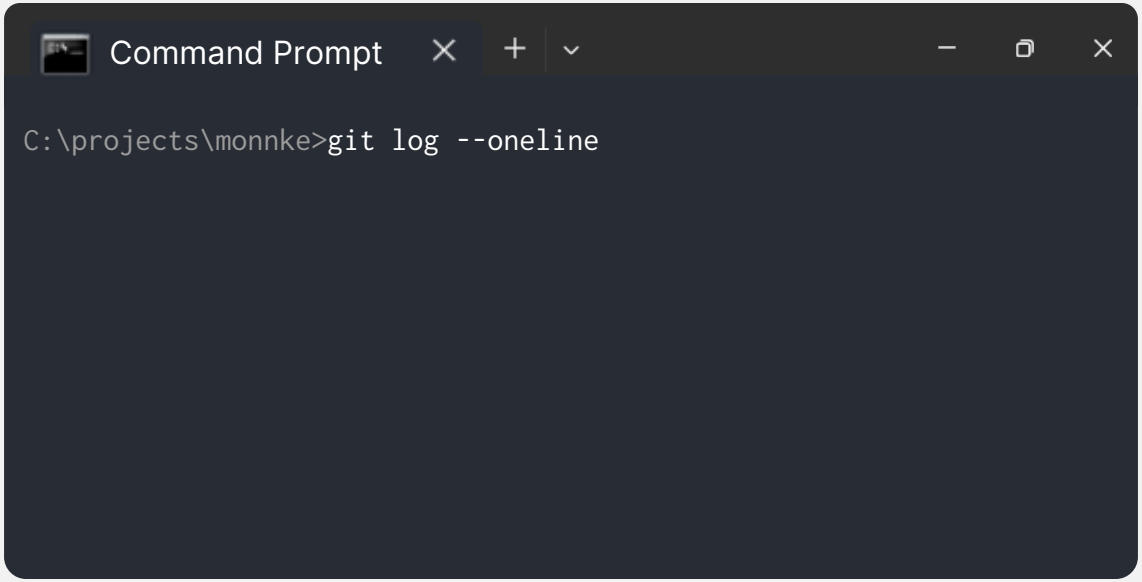
#7



NOTE: A text file will open, then edit, save, and close it to complete changes.

View Commit History

#8



NOTE: Remove `--oneline` flag to display detailed commit history.

Revert Commits (Safe)

#9

Command Prompt

```
C:\projects\monnke>git revert df6a800

C:\projects\monnke>git push
```

NOTE: Run ``git log`` and find the previous commit ID before the most recent commit. This creates a new commit that “undos” last commit, but not change the commit history.

Undo Commit with Sensitive Info

#10

Command Prompt

```
C:\projects\monnke>git rebase -i df6a800
### a file will open, change “pick” to edit, then close file...
### manually remove file or sensitive data in code...

C:\projects\monnke>git commit --amend git push --force
### a file will open, edit commit message, then close file...

C:\projects\monnke>git push origin MK010 --force
```

NOTE: Run ``git log`` and find the previous commit ID before the most recent commit.

Reset to Previous Commit

#11

Command Prompt

```
C:\projects\monnke>git reset --hard <commit-id>

C:\projects\monnke>git rebase -i a4e1c92
```

NOTE: This will discard all of you current changes.

Display Local Branches

#12

Command Prompt

```
C:\projects\monnke>git branch
* main

C:\projects\monnke>
```

NOTE: If diffs is too long, press `ENTER` to scroll or `q` to end.

Display All Branches

#13

Command Prompt

```
C:\projects\monnke>git -a branch
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main

C:\projects\monnke>
```

Create New Branch

#14

Command Prompt

```
C:\projects\monnke>git switch -c MK010
Switched to a new branch 'MK010’

C:\projects\monnke>
```

Switch to an Existing Branch

#15

Command Prompt

```
C:\projects\monnke>git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main’.

C:\projects\monnke>
```

Compare Unstaged to Last Commit

#16

Command Prompt

```
C:\projects\monnke>git diff
diff --git a/architectures.py b/architectures.py
index 1cbf56a..10e70d2 100644
--- a/architectures.py
+++ b/architectures.py
@@ -49,7 +49,7 @@ class ResNeXtBlock(Layer):


 def _load_efficient_net(width, height, channels):
-    """EfficientNet implementation"""
+    """ EfficientNet implementation """
```

NOTE: If diffs is too long, press `ENTER` to scroll or `q` to end.

Compare Staged to Last Commit

#17

Command Prompt

```
C:\projects\monnke>git diff --staged

C:\projects\monnke>
```

NOTE: If diffs is too long, press `ENTER` to scroll or `q` to end.

Compare HEAD to Remote

#18

Command Prompt

```
C:\projects\monnke>git diff origin/main

C:\projects\monnke>
```

NOTE: If diffs is too long, press `ENTER` to scroll or `q` to end.

Managing Repositories

Advanced Git Commands

Safe Rename Current Branch (-m)

#1

Command Prompt

```
C:\projects\monnke>git switch MK010

C:\projects\monnke>git branch -m MK010-bugfix

C:\projects\monnke>
```

NOTE: Avoid using `-M` as this will overwrite a conflicting branch.

Safe Deleting Local Branch (-d)

#2

Command Prompt

```
C:\projects\monnke>git switch main
Switched to a new branch 'main'
Your branch is up to date with 'origin/main'.

C:\projects\monnke>git branch -d MK010-bugfix
Deleted branch MK010-bugfix (was 6cdfc06).

C:\projects\monnke>
```

NOTE: Avoid using `-D` as this will force delete a any/merged branch.

Push Branch to Remote

#3

Command Prompt

```
C:\projects\monnke>git switch -c MK010-feature
Switched to a new branch 'MK010-feature'

C:\projects\monnke>git push -u origin MK010-feature
To github.com:shameni/monnke.git
 * [new branch]      MK010-feature -> MK010-feature
branch 'MK010' set up to track 'origin/MK010-feature'.

C:\projects\monnke>
```

Rename Remote Branch

#4

Command Prompt

```
C:\projects\monnke>git branch -m MK010-feature MK1

C:\projects\monnke>git push origin MK1
To github.com:shameni/monnke.git
 * [new branch]      MK1 -> MK1

C:\projects\monnke>
```

NOTE: Git hosting platforms typically introduce access controls to avoid rogue deletions and other accidental or malicious activities.

Deleting Remote Branch (--delete)

#5

Command Prompt

```
C:\projects\monnke>git switch main
Switched to a new branch 'main'
Your branch is up to date with 'origin/main'.

C:\projects\monnke>git push origin --delete MK1
To github.com:shameni/monnke.git
 - [deleted]         MK1
C:\projects\monnke>
```

NOTE: Git hosting platforms typically introduce access controls to avoid rogue deletions and other accidental or malicious activities.

Stash Uncommited Changes

#6

Command Prompt

```
C:\projects\monnke>git stash

C:\projects\monnke>git switch main
```

NOTE: When you need to urgently work in another branch. This is similar to a “Cut” operation.

Copy Uncommited Changes

#7

Command Prompt

```
C:\projects\monnke>git stash

C:\projects\monnke>git switch main

C:\projects\monnke>git stash apply

C:\projects\monnke>git switch MK010

C:\projects\monnke>git stash pop

C:\projects\monnke>
```

NOTE: It included both unstaged and staged changes. This is similar to a “Copy and Paste” operation.

Move Uncommited Changes

#8

Command Prompt

```
C:\projects\monnke>git stash

C:\projects\monnke>git switch main

C:\projects\monnke>git stash pop

C:\projects\monnke>
```

NOTE: It included both unstaged and staged changes. This is similar to a “Cut and Paste” operation.

Merge: Bypass Pull Requests

#9

Command Prompt

```
C:\projects\monnke>git switch main

C:\projects\monnke>git merge MK010
Updating 6cdfc06..bcee70f
Fast-forward
 architectures.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

C:\projects\monnke>git push origin main
...

C:\projects\monnke>
```

NOTE: Git hosting platforms typically introduce access controls to avoid rogue deletions and other accidental or malicious activities.

Create Tags

#10

Command Prompt

```
C:\projects\monnke>git tag "v1.0.9" 6cdfc06

C:\projects\monnke>git push origin -tags

C:\projects\monnke>git ls-remote --tags origin

C:\projects\monnke>
```

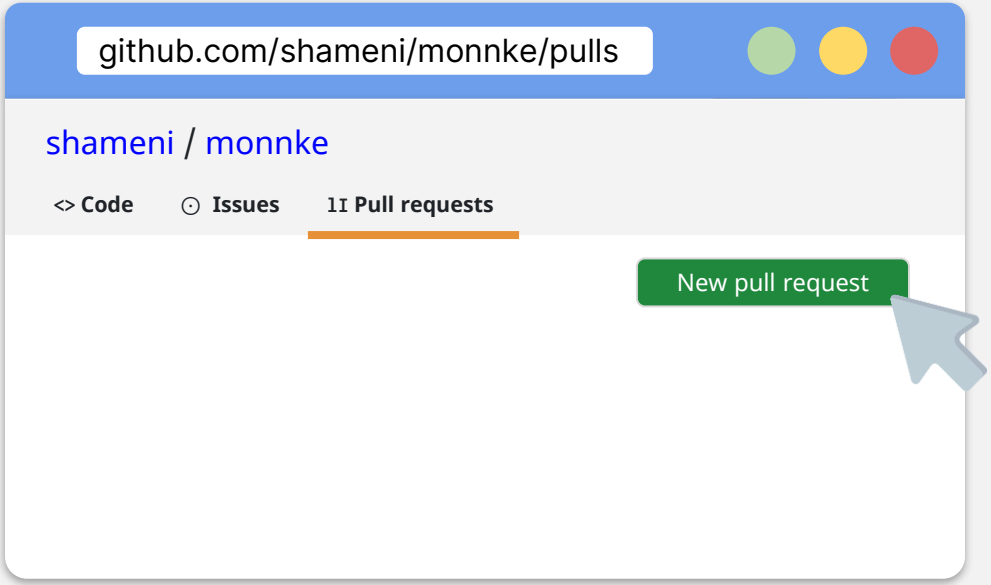
NOTE: Run ``git log`` and find the desired commit ID for tagging.

Managing Repositories

Pull Requests

Creating a Pull Request (PR)

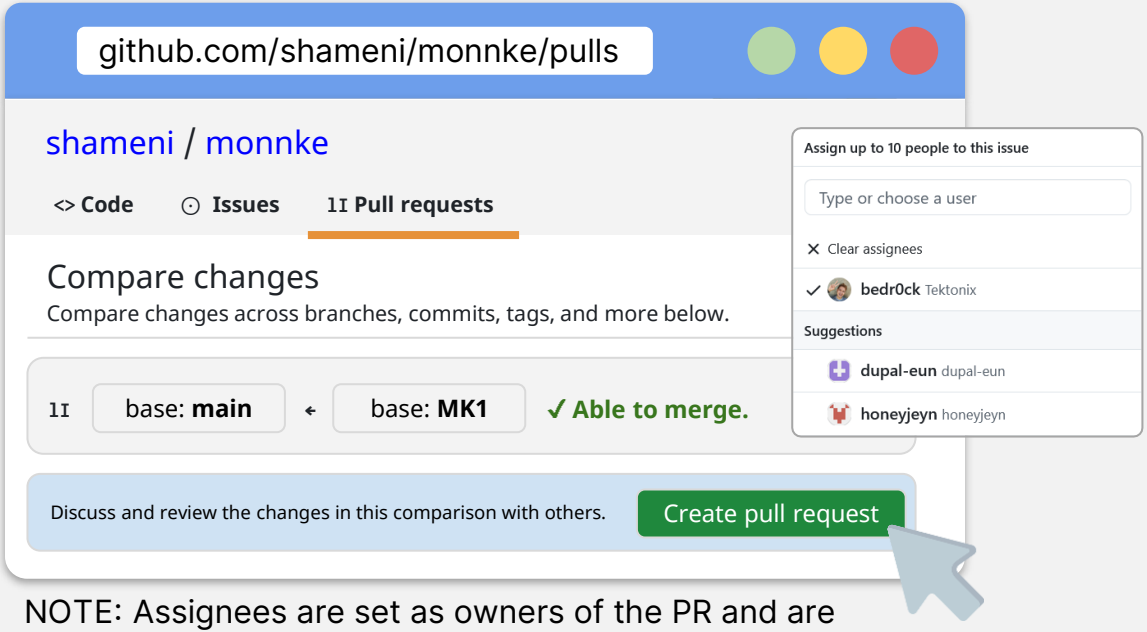
#1



NOTE: Other platforms uses “Merge Requests” and may have different internal mechanisms to perform code review and merging.

Creating a Pull Request (PR)

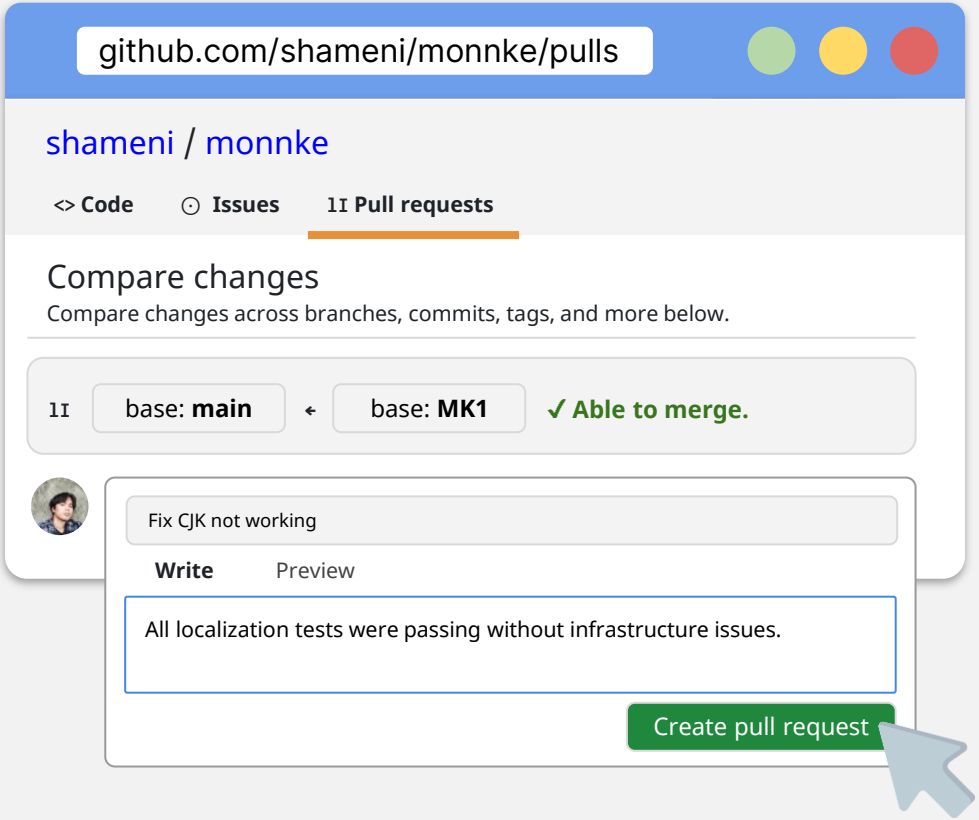
#2



NOTE: Assignees are set as owners of the PR and are responsible in assessing the request, but anyone with access can still work on that PR.

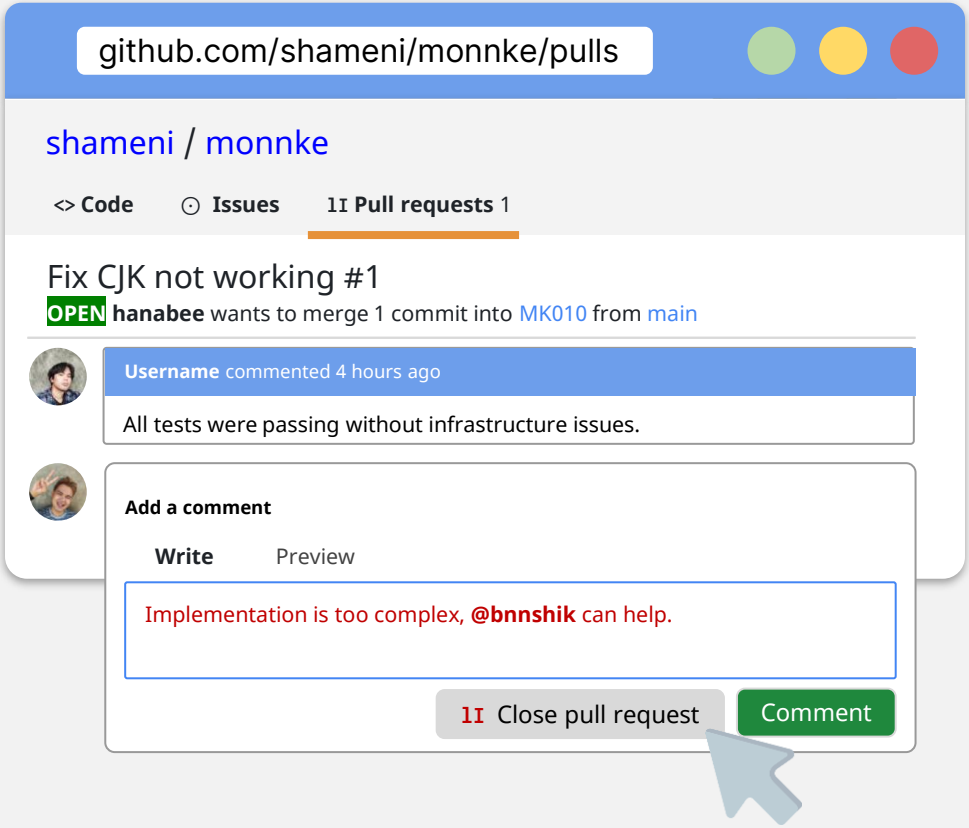
Creating a Pull Request (PR)

#3



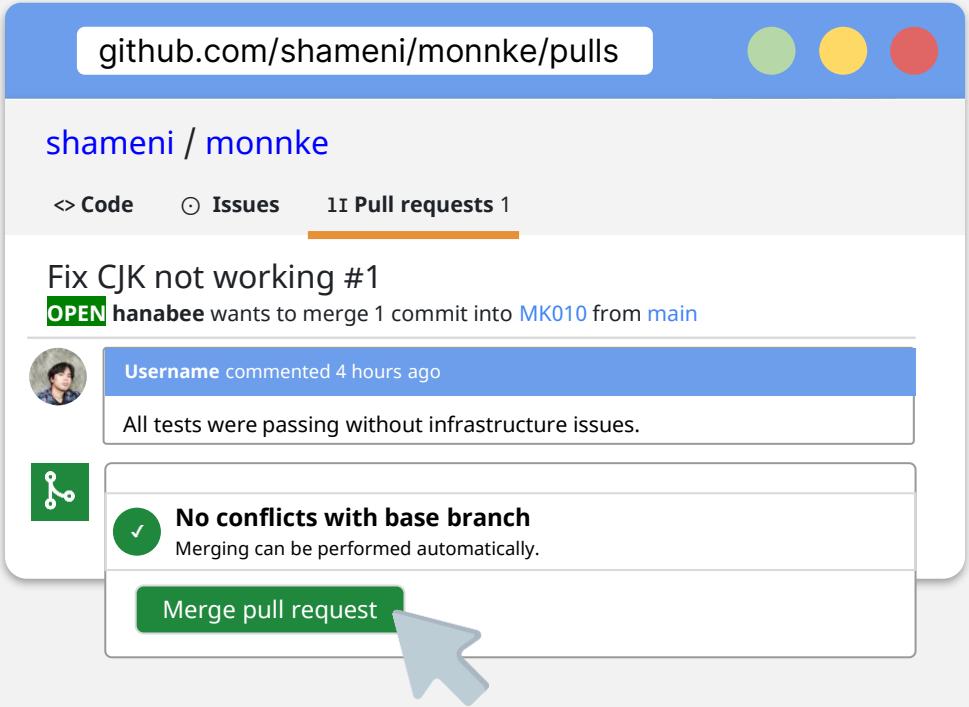
Creating a Pull Request (PR)

#4



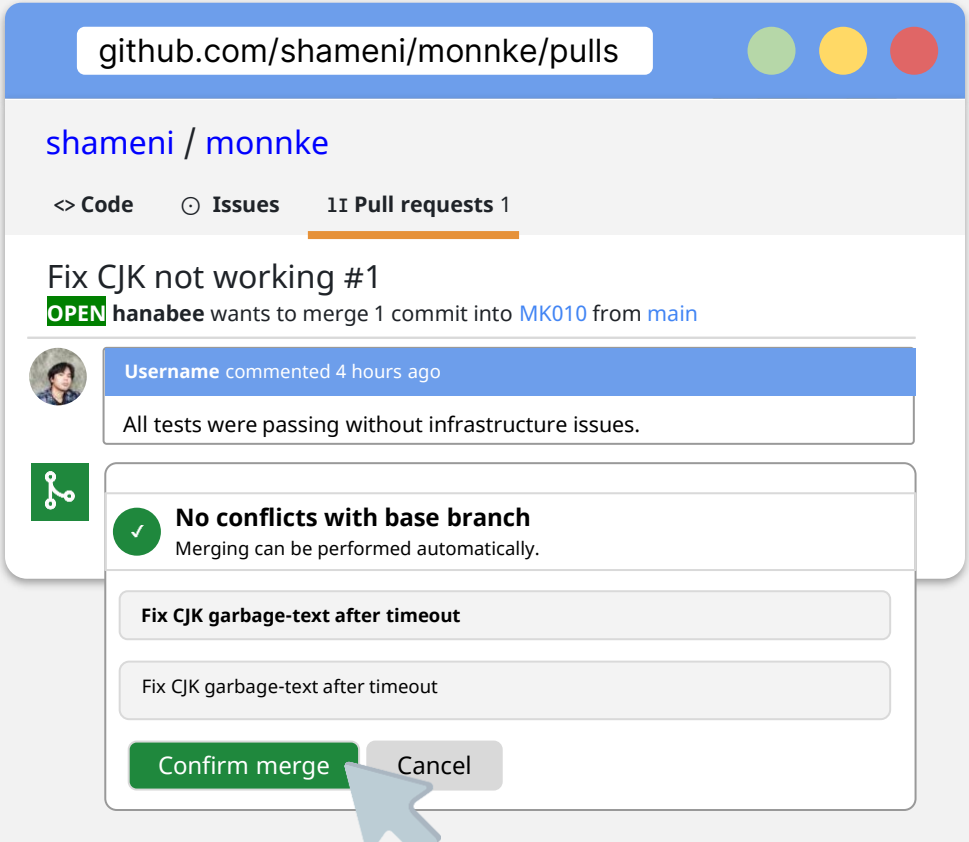
Creating a Pull Request (PR)

#5



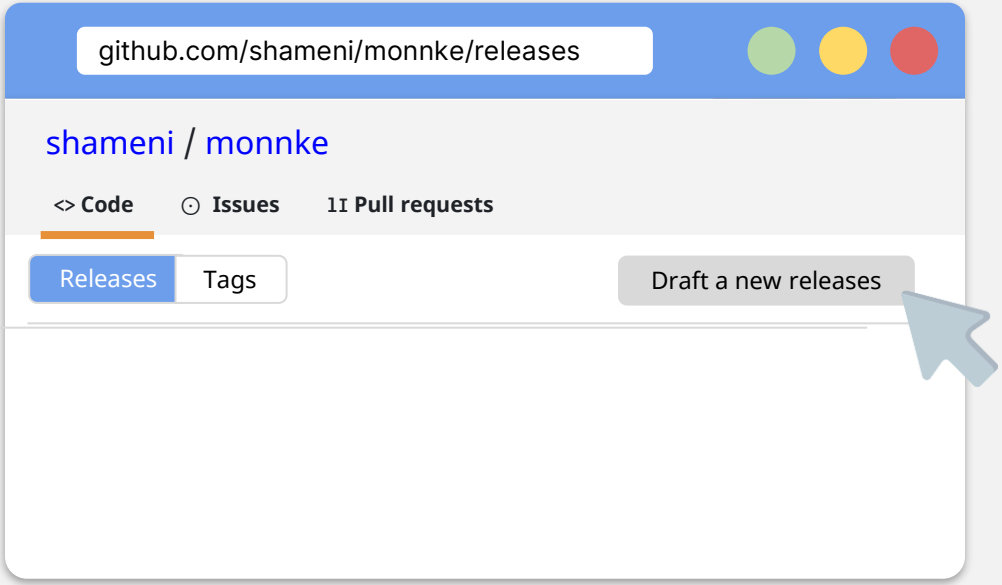
Creating a Pull Request (PR)

#6



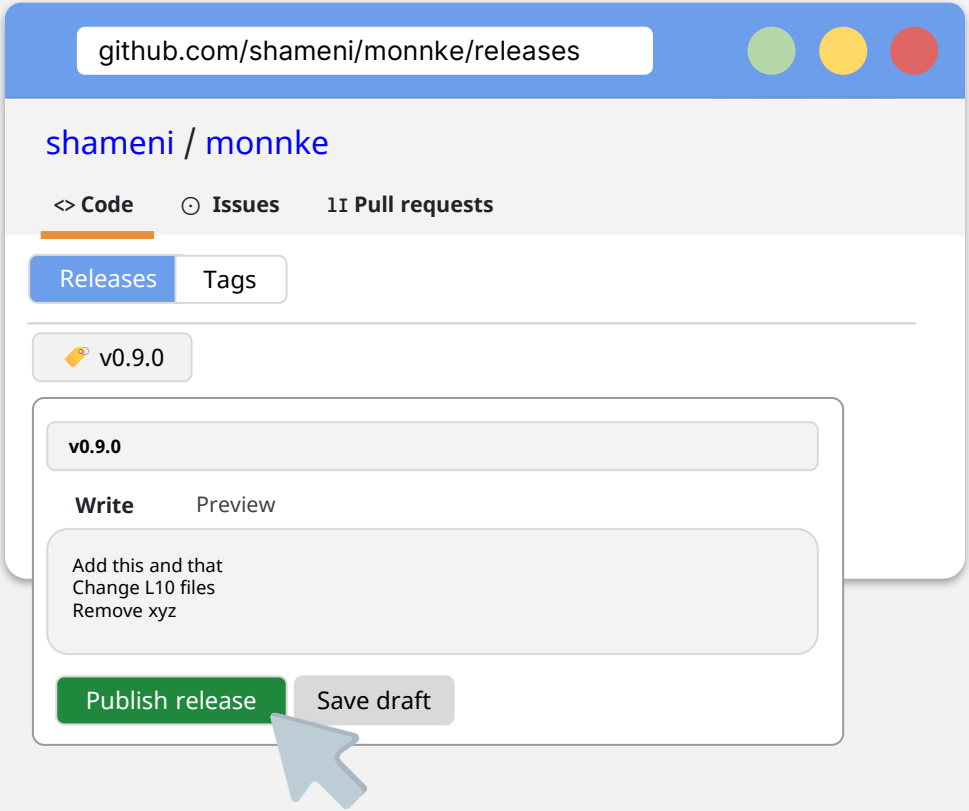
Creating Releases

#7



Creating Releases

#8



Managing Repositories

Commit Messages

What is a commit message?

It is a short textual description that explains the context and intent of a commit.

Who reads commit messages?

And why it is important to craft good messages...

Developers

Developer typically scan through sets of commits to integrate changes or investigate breakage.

Team Leads

Most cross-functional leads may scrape commit history for reporting purposes.

Testers (Rarely)

Testers may sometimes access commit history to gather information when logging bugs and other breakages,

Future Devs & You

In the future, you may be tasked to find how a bug came to be or why you even changed the code.

Commit Message Format

```
Concise summary of changeset

PROBLEM
DIAGNOSIS
SOLUTION
TESTING
REVIEWERS
BUG TRACKER ID
External References
Mechanical junk
```

Write in imperative and present mood, saying what the change does, not what the author did. Max=50 chars.

Avoid wall of text or overly-formatted body.

External servers may become unavailable and unrecoverable through time.

Technical keywords may become obsolete or unrecognizable.

Commit Message Format

```
Concise summary of changeset
(blank line)
Free-form prose of change and purpose,
as short as possible.
(blank line)
Necessary TL;DR, wall of text. (optional)
```

Wrap at 72 chars per line.
Focus on the motivation for the change,
not just what was changed.

General Guidelines

- Rigorous Testing: Double-check before pushing!

before

```
Fix typo in XYZ
```

after

```
# this commit should not even exist!
```

General Guidelines

- Rigorous Testing: Double-check before pushing!

before

```
Fix server still not working
```

after

```
# this commit should not even exist!
```

General Guidelines

Helpful Summaries: Make brief but informative.

before

```
D03916 Breakage=yes


```

after

```
Add pagination to product listing page

Product list queries all matches from the database, ensure `page` is set to `1`
by default. Introduced after migrating to new API route in release v1.9.12.

D03916: Browser page not working when searching products
```

General Guidelines

Prefix Subsystem: Identify isolated subsystem.

before

```
Fix garbage texts when editing bio


```

after

```
[l10n] Handle DBCS decoding on text inputs

CJK was not decoded, since non-English speakers were not anticipated. Ensure
DBCS characters are decoded and displayed properly through a common interface.

D00852: Unrecoverable garbage text when editing bio
```

General Guidelines

Decouple Changes: No monster changes!

before

```
(18462 lines changed in 32 files)


```

after

```
[FindBin] Refactor pathfinding algorithm for improved performance

Canonical A* is very slow, added heuristics to improve speed on mobile devices.

D28421: Soft-hang when adding ‘paper’ trash
# break unrelated changes into multiple commits to ensure each has specific goal.
```

Summary

- Adhering to systematic versioning guidelines allows development teams to enhance productivity and minimize technical misunderstandings.
- The innovations in VCS stems from the increased demand in collaborative efforts across distributed teams working large projects.
- Although git is packed with commands, you will only need a few especially when following best practices.
- Well-crafted commit messages allows current and future developers to efficiently investigate bugs even when existing teams and external references becomes unavailable.

Scan me!

Evaluation Form

forms.gle/TncdC5nEKJZYucVz6

Thank You!

References

- atlassian.com/git/tutorials/why-git
- javatpoint.com/git-vs-mercurial
- docs.github.com/en/get-started/using-git/about-git
- youtube.com/watch?v=4XpnKHJAok8
- simplilearn.com/tutorials/git-tutorial/what-is-git
- slideshare.net/IanSkerrett/eclipse-survey-2013-report-final
- survey.stackoverflow.co/2022/#section-version-control-version-control-systems
- git-scm.com/docs
- business-science.github.io/shiny-production-with-aws-book/git-command-line-interface-cli.html
- dineshonjava.com/difference-between-git-add-a-and-git-add-dot-and-git-u/
- code.visualstudio.com/docs/sourcecontrol/overview
- blogs.gnome.org/danni/2011/10/25/a-guide-to-writing-git-commit-messages/
- mediawiki.org/wiki/Gerrit/Commit_message_guidelines
- 365git.tumblr.com/post/3308646748/writing-git-commit-messages
- tbaggery.com/2008/04/19/a-note-about-git-commit-messages.html